

GlassBox

An explainable KPI root-cause engine -- README

Team Vantage -- Accenture Innovation Challenge 2026, Track 3

GlassBox

Every dashboard tells you *what* changed. GlassBox tells you *why*, shows you the evidence, and tells you how much to trust it — and says "I don't know" when the data doesn't support an answer.

A minimal 2-screen Streamlit app exists ([app/main.py](#) — scenario/persona picker, tiered findings view) and runs: `make app`. It's verified two ways — headlessly (every one of the 17 scenarios and all three persona overrides execute through the real UI with zero exceptions, Streamlit's `AppTest` harness) and visually (real screenshots of the rendered UI at every stage of a run — home screen, evidence drawer, abstention, a flagged prompt-injection document — in [screenshots/](#), not mockups). Everything else below is verifiable directly in this repo too: real code, a real committed dataset, and a scorecard you can regenerate yourself with no API key. [5-minute evaluation guide](#) · [Reproduce our results](#) · [Run traces](#) · [Improvement changelog](#)

The person this is for

Priya runs the Audio category at a mid-size electronics retailer. On Tuesday morning her dashboard shows net revenue in her category down sharply in the South — a real run of this system, on the committed dataset, measures it at -23.5% (-₹34,99,600) against the prior week ([full trace](#)). The dashboard tells her *that*. It cannot tell her *why*.

So she files a request. An analyst pulls the numbers, slices by region and channel, finds nothing conclusive, asks the supply-chain team, waits, reads forty support tickets, and comes back days later with: *"it looks like delivery times slipped in southern hubs."*

This is the bottleneck. Not the dashboard — dashboards are good at what they do. The translation from *what changed* to *why*, and *what to do* is manual and it finishes well after the window in which it mattered. (We don't have primary research on exactly how many investigations a month this costs a retailer like Priya's employer — that number needs a real customer conversation, not an invented statistic, so we're not printing one here.)

What GlassBox changes: the same investigation, run automatically, with every claim graded and every source cited. On the run above, GlassBox found the same underlying cause — a South-hub courier capacity cut — cited four support tickets and a field report, and *also* found and correctly rejected a second, genuinely plausible cause (a national promotion that ended the same week): it survived its own significance test at the national level but failed when checked against South specifically, because the promotion ended everywhere and South's drop was larger than that alone explains. [See the full seven-stage trace](#).

And when the data genuinely doesn't contain an answer, it says so — which is the part that took the longest to build. [See SC-08](#) — one of our 17 pre-registered test scenarios ([SC-01...SC-17](#), each a distinct planted-cause or no-cause case defined in [data/injection_manifest.yaml](#) and described in full in [docs/03_SCENARIOS.md](#)); this one is designed with no real cause planted at all.

([See screenshots/](#) for the real rendered UI on this exact scenario — headline and tier badges, the evidence drawer open on the cited ticket, the rejected rival cause. The trace above is the same findings object in the format the engine actually produces it; `make app` renders it live if you want to run it yourself.)

Priya is the user. The buyer is one level up: the CDO or analytics platform owner, sold one design partner at a time — a layer on the BI stack the enterprise already owns, not a rip-and-replace. Full case, including what we're not willing to guess a price on: [docs/09 BUSINESS PROPOSAL.md](#).

How it works

Seven stages. **Five of them never touch a language model.**

question or alert

↓

00 INTENT ♦ text → {kpi, segment, window}, from contract vocabulary only

- 01 DEFINE • the metric's governed definition → SQL → series
 - 02 DETECT • seasonal baseline • is this signal or noise?
 - 03 LOCALIZE • which segment actually moved
 - 04 DECOMPOSE • price vs volume vs mix
 - 05 RETRIEVE • tickets, CRM notes, field reports – who already noticed
 - 06 FALSIFY • try to disprove each candidate cause
 - ↓ gate: answer | abstain | clarify
 - 07 NARRATE • write it, with every number validated against the computation
- deterministic • the only two model calls

The model turns a question into a query, and a finished result into a sentence. **It never decides what is true.** Every number in the output is re-extracted and matched against the computed findings before it reaches a human; unmatched output is regenerated, then falls back to a template.

Every sentence carries a confidence tier:

VERIFIED	Recomputed from source. Arithmetic, not judgement.
EVIDENCED	Supported by specific documents we can show you.
TESTED	We tried to disprove this and failed.
CORRELATED	Moves together with the metric. Not established as a cause.
HYPOTHESIS	A plausible lead with no support yet.
UNKNOWN	We are not willing to grade this.

Tiers are assigned by a rule table, never by the model. Removing evidence can never raise one — that property is unit-tested.

Does it work?

Zero hallucinated causes across 15 scored scenarios. Zero. Not "low" — zero, measured, with the misses published alongside the passes. Here's the full picture, and exactly how to reproduce every number in it yourself.

Quoted from the committed [eval/scorecard.md](#) — regenerate it yourself with `GLASSBOX_REPLAY=1 python -m eval.harness --scenarios all --out eval/scorecard.md`, no API key needed, and it will reproduce these numbers exactly. This isn't a theoretical claim: the committed scorecard was produced by a real live run ([CHANGELOG.md](#) entries 019-028), and a fresh `GLASSBOX_REPLAY=1` regeneration diffs byte-for-byte identical against it, aside from the header commit hash.

17 pre-registered scenarios, 15 scored, on data where we planted the causes, so we know the right answers. Two (SC-07: two co-equal causes; SC-15: localizes to a below-disclosure-threshold cell) are retired in place, per our own pre-registration rule for an ill-posed scenario — full reasoning in [data/manifest_reconciliation.md](#); both remain visible in the manifest and the scorecard's own **RETIRED** section, not hidden:

GlassBox	
Exact-match scenarios passed	7 / 15
Root cause, top-1 (of scenarios with a segment to match)	2 / 11
Hallucinated causes	0 / 15
Abstention recall (caught the negative control)	1.00 (1/1)
Abstention precision	0.33 (1/3)

B3 (single-LLM-prompt) isn't in this table because it isn't scored the same way GlassBox is — it

doesn't investigate a segment, it just talks. But it has been run live (OpenAI, [gpt-4o-mini](#), same retrieved evidence GlassBox saw, one prompt, no pipeline — after Groq's account-level quota and then Gemini's rate limit both blocked completion, [CHANGELOG.md](#) entries 026-027) across 14 of the 15 scored scenarios, and the real, unedited transcript is in [eval/baseline_scorecard.md](#). B1 (naive drill-down — rank the biggest single-dimension segment, cite the most recent matching ticket, no falsification) is also real and run. Headline result: of the 3 scenarios where no cause was planted (SC-02, SC-08, SC-17), B1 asserts a confident cause on all 3, and B3 asserts one — unhedged, in fluent prose — on the 2 of those 3 it was actually sent a prompt for (SC-17 is a clarification scenario with no single question to hand it). GlassBox's rate on the same 3 scenarios: 0/3.

The 8 non-exact scenarios split two ways, neither of which is a fabrication: 2 abstain conservatively where an answer was possible (the system declining rather than guess), and 6 answer correctly on the right branch, with real cited evidence, but name a broader or adjacent segment than the exact ground truth. [eval/scorecard.md](#)'s Misses section has the specific reason for each.

On real data: the RS benchmark (135 real anomalies against seven published localization algorithms) is planned but has not been run — [eval/rs_benchmark.md](#) says so honestly rather than showing invented numbers.

The primary dataset (Meridian) is synthetic — because the real one doesn't exist. No public dataset pairs business KPIs, customer text, and labelled root causes — we had to build the evaluation because the field never had one ([docs/05_DATA_STRATEGY.md](#) §2 makes the full case). The generator ([data/generate.py](#)) is built to make this hard, not convenient: ramped changes instead of clean steps, partial spillover into the "control" segments so they aren't pristine, a deliberately confounded rival cause (SC-01's national promo, timed to end the same week). Real-data benchmarks (RS, PSqueeze) sit alongside it — see [docs/05_DATA_STRATEGY.md](#) for what's used, why, and under which licences.

The ground truth was frozen before the engine existed. Commit [c908ef9bd569befc754c8b28d1db99b4ba590f52](#) is the repo's first commit and contains [data/injection_manifest.yaml](#), before any file under [engine/](#) exists. Check it: `git log --follow data/injection_manifest.yaml`.

The scorecard includes our failures, with an explanation for each. Some of them are the system behaving correctly under a hard case; we say which.

The result we care about most

On **SC-08**, a real run against the committed dataset finds a real, material -13.3% drop ($z = -8.06$ — statistically unambiguous) and **no planted cause**. The corpus is salted with plausible-sounding decoys. GlassBox investigates, finds nothing that survives a falsification test, and says so:

*"Net Revenue moved -13.3% in the window to 2026-07-26, but no cause could be established that clears the evidence floor." Ruled out: a Web-channel localization that failed its own control-segment check. Referred to Analytics, with the specific reason recorded. **Zero causes asserted.***

The obvious next step — the same data and retrieved documents through a single LLM prompt, to see whether it invents a cause where we don't — is exactly what we ran, live (OpenAI, [gpt-4o-mini](#)), on this same scenario. Its real, unedited answer:

"The observed decline in net revenue can likely be attributed to a combination of factors, particularly issues with product returns due to packaging problems and a shift in the app's order composition, which saw fewer high-value items being purchased... I am reasonably confident in this assessment, given the statistical significance of the movement and the relevant evidence retrieved."

Fluent, specific, confident — and wrong. There is no planted cause in this window; the evidence it's citing is exactly the same decoy corpus GlassBox saw and declined to act on. This is the comparison the whole project rests on: same evidence, same retrieval, one system invents a confident story and the other says it doesn't know. Full transcript in [TRAJECTORIES.md](#)'s §2 and [eval/baseline_scorecard.md](#); [JUDGES.md](#) for how to run this yourself.

Trust and deployment

GlassBox ships to the data, never the reverse. In production it runs inside the customer's own cloud tenant, reads their warehouse through a read-only service account, and reaches the model

through a private endpoint in their own subscription. Five of seven stages never touch a model; the model sees a minimised, redacted findings object and snippets of documents the asking user was already entitled to read.

Demonstrated in the prototype, not just claimed — no UI picker yet, so these are traced directly rather than clicked:

- **Prompt injection** — a malicious support ticket is retrieved, quoted as evidence, flagged, and ignored. Found doing this in two different scenarios, one of them unplanned. ([SC-14 trace](#))
- **Small-cell suppression** — a movement localizing below the minimum cell size is rolled up to a disclosable level so no individual is inferable ([SC-15; engine/entitlements.py::suppress_small_cells](#))
- **Role-based access** — different personas produce a different SQL predicate and a different `entitlements_hash` for the same question — shown directly, no UI needed, in [JUDGES.md's 15-minute path](#)
- **Audit log** — every run: who asked, what was computed, what was withheld ([engine/audit.py](#), append-only JSONL)

Details in [docs/06 SECURITY TRUST.md](#).

Run it

```
git clone <this repo> && cd glassbox
make reproduce          # setup → generate data → run 15 scored scenarios → tests
```

`make app` (or `streamlit run app/main.py`) launches the UI — scenario picker, persona switcher, tiered findings view. No `make` on your machine (Windows without WSL, for instance - try `mingw32-make` first if you have MinGW/MSYS installed for some other reason)? The four reproduce steps are just:

```
pip install -r requirements.txt
python data/generate.py --seed 20260829 --out data/generated
GLASSBOX_REPLAY=1 python -m eval.harness --scenarios all --out eval/scorecard.md
pytest -q
```

Works fully offline, no API key required by default (`GLASSBOX_REPLAY=1` in `.env.example`, and `make eval/make reproduce` force it in the recipe regardless of what's in your `.env` - a judge can never trigger a billed call by running the promised command). `eval/replay_cache/` now holds real, live-recorded responses for every scored scenario, so a fresh clone with no key resumes those exact real narrations; only a scenario with no cached entry at all falls back to the deterministic parser / template narrator — see [TRAJECTORIES.md](#) for exactly what that fallback output looks like. Every findings object validates against the schema and passes the number validator either way. Full guide, expected outputs and troubleshooting: [REPRODUCE.md](#).

Want genuinely live output, not replay? Groq's free tier ([console.groq.com/keys](#) — about a minute to sign up, no card needed) is enough for a full run: in `.env` set `GLASSBOX_REPLAY=0`, `GLASSBOX_LLM_PROVIDER=groq`, `GLASSBOX_LLM_MODEL=openai/gpt-oss-120b` (the model this repo's own live captures used — see [CHANGELOG.md](#)), and `GLASSBOX_LLM_API_KEY` to your key, then `make app` and run any scenario. The UI's badge switches from `REPLAYING` to `LIVE model call`, and what you're reading was generated on the spot.

Four provider adapters exist in `engine/llm_client.py`: Anthropic, Groq, Gemini, and OpenAI. All four are wired the same way — swap `GLASSBOX_LLM_PROVIDER` in `.env` and nothing else changes. All three non-Anthropic adapters have been run live this session: Groq (`openai/gpt-oss-120b`) and Gemini (`gemini-3.6-flash`) produced the real narrations in `eval/replay_cache/` and [TRAJECTORIES.md](#) (12 Groq + 7 Gemini captures, after Groq's account-level free-tier quota blocked further calls); OpenAI (`gpt-4o-mini`) produced the B3 quotes in `eval/baseline_scorecard.md` after Gemini's rate limit also blocked completion. Three different provider-specific quirks were found and fixed live, not simulated — see [CHANGELOG.md](#) entries 022-028. The Anthropic adapter is implemented but hasn't been run live in this repo.

What's in here

engine/	the pipeline – one file per stage
contracts/	semantic contracts: definition + lineage + access + drivers
prompts/	both prompts, versioned – the only two model calls
data/	generator + the frozen injection manifest (ground truth)
eval/	harness, metrics, baselines, committed scorecards
app/	Streamlit UI – thin, ~20% of the effort; reads findings objects the engine
already computed	
docs/	architecture, evaluation plan, data strategy, ADRs

Our hot take

The failure mode of analytics AI is fluency, not accuracy.

A model asked why revenue fell will always produce a fluent, plausible, well-structured answer — including when the data contains no answer at all. And fluency is indistinguishable from correctness at reading speed, which is exactly the speed at which a business decision gets made. The dangerous output is not the one that is obviously wrong. It is the one that is confidently, articulately wrong about a number someone is about to act on.

So we built the system to be *capable of declining*, and then measured how often it declines when it should. Our hardest engineering problem was not finding causes. It was building something that would stay quiet.

And our honest limitation: GlassBox's causal claims rest on quasi-experimental tests over observational data. Difference-in-differences against imperfect control segments is suggestive, not proof. The tiers are honest about this — **EVIDENCED** and **TESTED**-equivalent mean "survived our falsification attempts", not "proven" — but a confounder that moves with the true cause across every control segment will defeat it. **SC-12 is that scenario** (spillover contaminates the would-be control segments) — in the current build it abstains rather than force an answer with degraded confidence, which is a stricter response than we originally designed for and not the one we'd ultimately want, but it is still the system declining to overclaim rather than guessing. [eval/scorecard.md](#)'s Misses section has the exact reason. That's the ceiling of what this method can claim, stated as it actually behaves today, not as we intended it to.

Team Vantage. MIT licensed (see [LICENSE](#)). External benchmark datasets used under their own licences where applicable: RiskLoc (MIT), PSqueeze (CC BY 4.0) — see [data/README.md](#). No Olist or other CC BY-NC-SA data is committed to this repo, by design (see [docs/05_DATA_STRATEGY.md](#) §3).